Лабораторна робота №3

Розробка серверної частини (Back-end) та API

Студент: Бабенко Олександр Олександрович

Група: 402-ТН

Викладач: Тютюнник Петро Богданович

1. Посилання на Git-репозиторій

https://github.com/Radonskiy/TRVD_2026_402TN_Babenko_Labs

2. Скріншоти роботи API

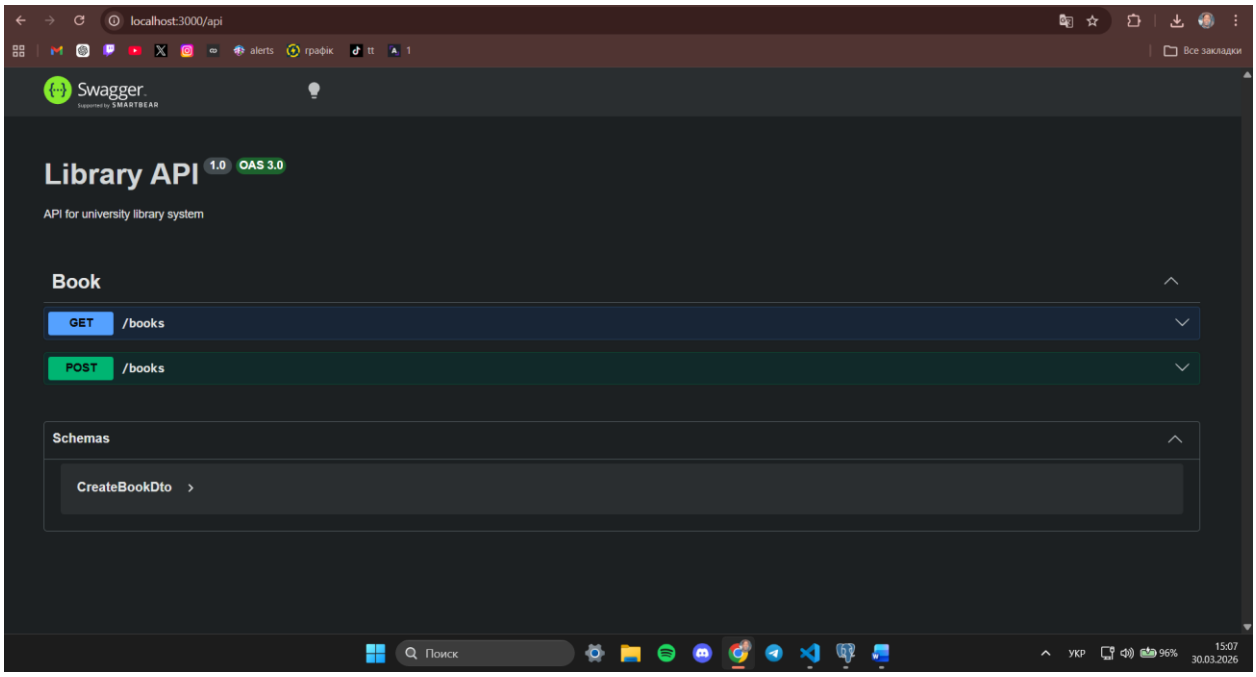

Рисунок 1 – Інтерфейс Swagger з доступними endpoint'ами

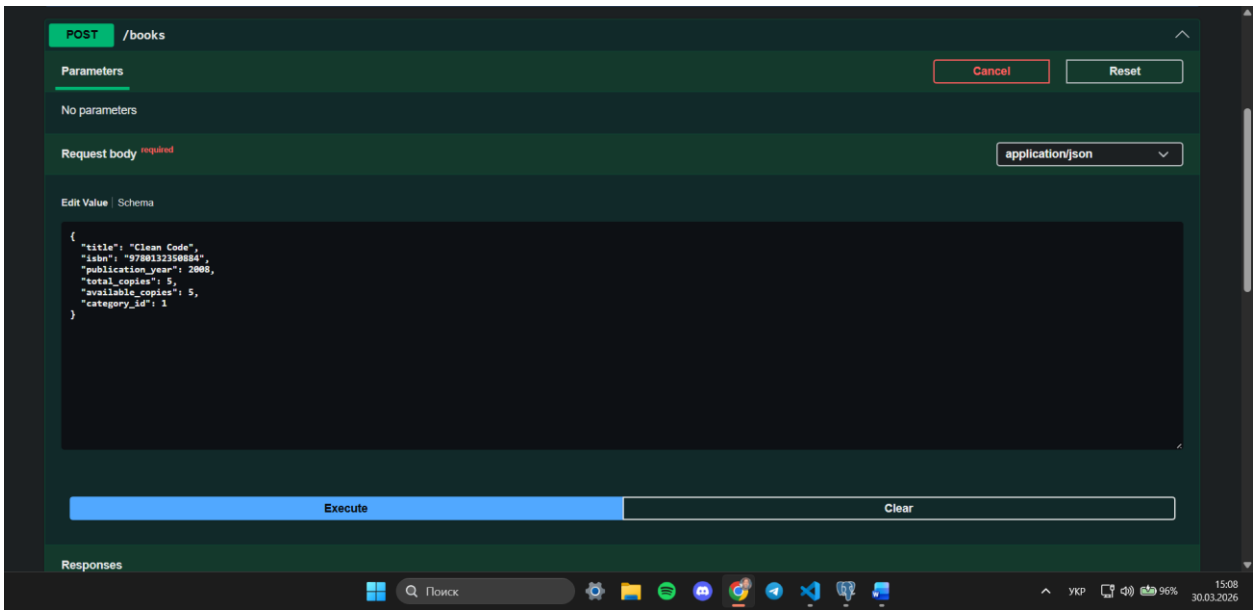

Рисунок 2 – Формування POST-запиту для створення книги

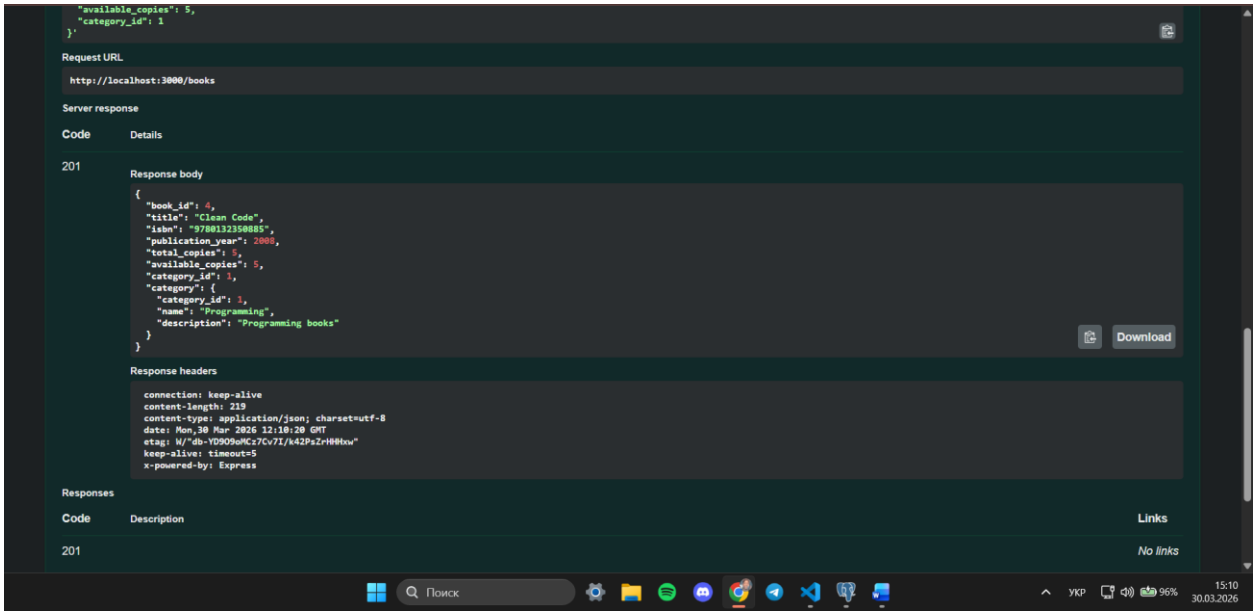

Рисунок 3 – Відповідь сервера після створення книги (201 Created)

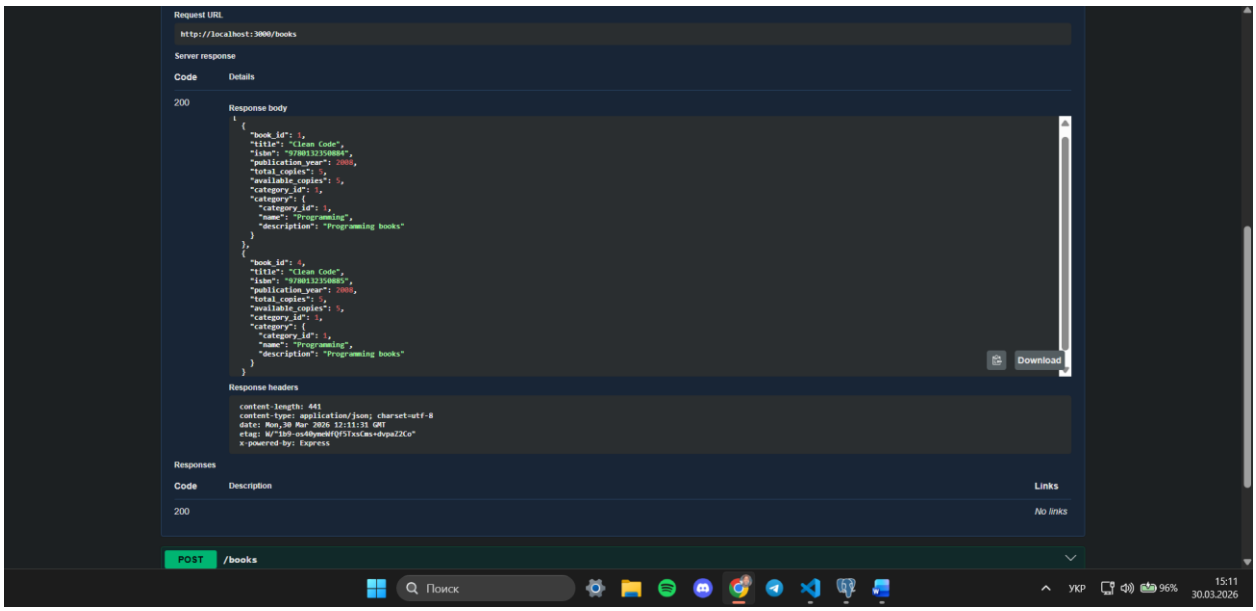

Рисунок 4 – Отримання списку книг через GET-запит

3. Приклади коду

Було реалізовано REST API для роботи з сутністю "Книга"

Лістинг 1 – DTO для створення книги

```
export class CreateBookDto {
  title: string;
  isbn: string;
  publication_year: number;
  total_copies: number;
  available_copies: number;
  category_id: number;
}
```

Лістинг 2 – Контролер для обробки запитів

```
@Controller('books')
export class BookController {
  constructor(private readonly bookService: BookService) {}

  @Get()
  findAll() {
    return this.bookService.findAll();
  }

  @Post()
```

```
  create(@Body() dto: CreateBookDto) {
    return this.bookService.create(dto);
  }
}
```

Лістинг 3 – Бізнес-логіка сервісу

```
async create(dto: CreateBookDto) {
  return this.prisma.book.create({
    data: dto,
  });
}
```

КОНТРОЛЬНІ ЗАПИТАННЯ

1. Порівняйте REST, GraphQL та gRPC

REST — це архітектурний стиль, який використовує HTTP-запити (GET, POST, PUT, DELETE). Він простий у реалізації та широко використовується.

GraphQL — це мова запитів до API, яка дозволяє клієнту отримувати тільки ті дані, які йому потрібні. Це зменшує обсяг переданих даних і підвищує ефективність.

gRPC — це високопродуктивний фреймворк від Google, який використовує Protocol Buffers і HTTP/2. Він забезпечує швидку взаємодію між сервісами.

Висновок:

- REST — для простих і стандартних API
- GraphQL — для гнучких клієнтських запитів
- gRPC — для високопродуктивних систем і мікросервісів

---

2. Що таке Over-fetching та Under-fetching

Over-fetching — це ситуація, коли API повертає більше даних, ніж потрібно клієнту.

Under-fetching — це ситуація, коли API не повертає достатньо даних, і клієнт змушений робити кілька запитів.

GraphQL вирішує ці проблеми тим, що клієнт сам визначає, які поля йому потрібні, і отримує тільки ці дані за один запит.

---

3. Що таке Protocol Buffers (Protobuf)

Protocol Buffers — це бінарний формат серіалізації даних, який використовується в gRPC.

Переваги:

- компактніший за JSON
- швидше обробляється
- строго типізований
- економить мережевий трафік

4. Як забезпечується версіонування API

У REST версіонування реалізується через URL, наприклад:

/api/v1/books
/api/v2/books

У GraphQL зазвичай не використовують версії. Замість цього:

- додають нові поля
- не видаляють старі одразу
- використовують позначку deprecated

---

5. Що таке Idempotency

Ідемпотентність — це властивість операції, при якій багаторазове виконання дає той самий результат, що й одне виконання.

Приклади:

- GET — ідемпотентний
- PUT — ідемпотентний
- DELETE — ідемпотентний
- POST — не є ідемпотентним

---

6. Роль SDL у GraphQL та .proto у gRPC

SDL (Schema Definition Language) у GraphQL використовується для опису структури API: типів, запитів і мутацій.

.proto файли у gRPC описують:

- структуру повідомлень (даних)
- доступні методи сервісу

Вони потрібні для генерації коду і забезпечення чіткої структури API.

Висновок

У ході виконання лабораторної роботи було розроблено серверну частину програмного забезпечення з використанням фреймворку NestJS. Було налаштовано підключення до бази даних PostgreSQL за допомогою Prisma ORM.

У процесі роботи реалізовано архітектуру застосунку з розділенням на шари: контролери, сервіси та шар доступу до даних. Створено REST API для роботи з сутністю "Книга", яке підтримує основні операції: створення та отримання даних.

Для тестування та документування API було використано інструмент Swagger, що дозволило перевірити коректність роботи запитів і відповідей сервера.

У результаті роботи отримано повноцінний серверний застосунок з реалізованим API, який може використовуватись як основа для подальшої розробки клієнтської частини системи.